

Software Engineering COMP1035

Lecture 10

Git Repository Platform Features





Today Learning Topics

1. Good Commit Messages
2. Issues
3. Labels
4. Milestones



Commit Message Conventions

Commit Message Components

- Ideally for **subject line**:
 - A concise summary of the change (50 characters or less).
 - Use the imperative mood (e.g. “Add feature”, NOT “Added feature”).
 - Capitalise the first word.
 - Do not end with a period.
- Ideally for **body message**:
 - More detailed explanation of the change.
 - Explain **why** the change was made, not just what was changed.
 - 70 characters or less.
 - Use bullet points or numbered lists for clarity.

Commit Message Components

- Footer (Optional):
 - For metadata or closing issues.
 - Example 1: `Signed-off-by: John Doe <john.doe@example.com>`
 - Example 2: `Closes: #456`

Commit Message Best Practices

- Use imperative mood: “Fix bug” (not “Fixed bug” or “Fixes bug”).
- Be specific: Instead of “Fix bug”, use “Fix login form validation bug”.
- Focus on the “Why”.
- Keep it concise.
- Follow a consistent style.
- Reference issues and pull requests: Use keywords like “Fix”, “Close”, “Resolve”.

Commit Message Types

- `feat`: A new feature.
- `fix`: A bug fix.
- `docs`: Documentation changes.
- `style`: Changes that do not affect the meaning of the code (whitespace, formatting, etc.).
- `refactor`: A code change that neither fixes a bug nor adds a feature.
- `perf`: A code change that improves performance.
- `test`: Adding missing tests or correcting existing tests.
- `build`: Changes that affect the build system or external dependencies.
- `ci`: Changes to CI configuration files and scripts.
- `chore`: Other changes that don't modify src or test files.
- `revert`: Reverts a previous commit.

Example Commit Message

```
git commit -m "feat: Add user profile page"
```

This commit adds a new user profile page that displays user information.

- Fetches user data from the API.
- Implements a responsive layout.
- Adds unit tests for the profile component.

Fixes #123"

```
team-sheldon % git commit -m "feat: Add user profile page"

This commit adds a new user profile page that displays user information.
- Fetches user data from the API.
- Implements a responsive layout.
- Adds unit tests for the profile component.

Fixes #123"
```




Git Repository Platform Features

Introduction to Git Platform Features

- Git handles version control and hosting platforms provide additional tools for project management and collaboration.
- Typically, Labels, Milestones, and Issues as key features offered by platforms like GitHub, GitLab, Bitbucket, etc.
- These features are essential for:
 - **Organization:** Categorizing and filtering work.
 - **Tracking Progress:** Monitoring progress towards goals and deadlines.
 - **Communication:** Centralizing discussions and task management.

Issues

- Issues are the fundamental building blocks for tracking work, reporting problems, and fostering discussions around your project.
- Think of issues as digital sticky notes or tasks that need attention.
- Issues provide a centralized and transparent way to manage everything from small bugs to large feature implementations.

Title (required)

Type

Issue

Description

Preview **B** *I* S | |

Write a description or drag your files here...

A more detailed explanation of the issue, such as reproduce (for bugs), acceptance criteria (for features), or any other relevant information.

Switch to rich text editing

Add [description templates](#) to help your contributors to communicate effectively!

☐ This issue is confidential and should only be visible to team members with at least Reporter access.

Assignee

Unassigned

Assign to me

Due date

Select due date

Milestone

Select milestone

A target timeframe or goal (e.g., a specific release) that the issue is associated with.

Labels

Select label

Categorization tags that help organize and filter issues.

Create issue

Cancel

The person or people responsible for working on or addressing the issue.

Issues

Activity

Sort or filter ▾

Preview

B

I





















































Write a comment or drag your files here...

A threaded discussion area where team members can ask questions, provide updates, share findings, and collaborate on the issue.

Switch to rich text editing



- **State:** The current status of the issue, typically including:
 - **Open:** The issue requires attention and is not yet resolved.
 - **Closed:** The issue has been resolved or addressed.

Open 1

Closed 0

All 1



Search or filter results...

 **Test**

#1 · created in 4 hours by BOON GIIN LEE

Example Scenario: Reporting and Fixing a Bug

- Let's imagine we have a web application, and a user reports a bug where the login button doesn't work on a specific browser.

1. Issue Creation:

- Title:** Login button not working on Safari browser
- Description:** 

```
## Bug Report: Login button not working on Safari
```

****Steps to Reproduce:****

1. Open the web application in Safari browser (version 16.x).
2. Navigate to the login page.
3. Enter valid username and password.
4. Click the "Login" button.

****Expected Behavior:****

The user should be redirected to their dashboard.

****Actual Behavior:****

The button appears to do nothing. No error message is displayed.

****Environment:****

- Browser: Safari 16.x
- Operating System: macOS Ventura

****Severity:**** High - Users cannot log in on Safari.

Example Scenario: Reporting and Fixing a Bug

2. Triage and Discussion:

- The development team reviews the issue.
- A developer might ask for more information in the comments:

“Developer A: Thanks for the detailed report! Could you check if there are any errors in the browser's developer console when you click the button?”
- The user might respond with a screenshot of the console showing a JavaScript error.

Example Scenario: Reporting and Fixing a Bug

3. Assignment and Labeling:

- The team decides this is a high-priority bug. They assign the issue to a developer, "Alice", and add the following labels:
 - `type:bug`
 - `priority:high`
 - `component:frontend`
 - `browser:safari`
- They might also assign it to the current `v1.1` Release milestone.

Example Scenario: Reporting and Fixing a Bug

4. Work in Progress:

- Alice checks out a new branch from the `main` branch, perhaps named `fix-safari-login`.
- She investigates the JavaScript error reported and identifies the cause of the issue (e.g., a browser-specific compatibility problem in the login script).
- She then modifies the code to fix the bug.

Example Scenario: Reporting and Fixing a Bug

5. Resolution and Pull Request:

- Once Alice has fixed the issue, she commits her changes and pushes the branch to the remote repository.
- She then creates a new pull request with the following details:
 - **Title:** Fix: Login button not working on Safari
 - **Description:**

“Fixes an issue where the login button was not functioning correctly on Safari browser. This was due to a compatibility issue with `displayMessage.js`. The code has been updated to handle this case.

This pull request addresses issue #123.”

Example Scenario: Reporting and Fixing a Bug

6. Review:

- Other developers review Alice's code in the pull request, ensuring the fix is correct and doesn't introduce any new issues.
- They might leave comments with suggestions or approvals.

7. Review: Merge:

- After a successful review, the Pull Request is merged into the main branch.

8. Closure: (automated or manually)

- Alice has included "Fixes #123" in the pull request description, so the issue with the number 123 (our login bug) is automatically closed upon merging the pull request.

Benefits of Using Issues

- All tasks, bugs, and discussions are **in one place**, making it easy to see what needs to be done.
- Issues provide a **dedicated space for discussing** specific topics, reducing the need for scattered emails or messages.
- Everyone on the team (and sometimes even external contributors) **can see the progress and status of different tasks**.
- Issues serve as a historical record of problems, solutions, and decisions made during the project.
- Labels and milestones help to categorize, prioritize, and track work towards specific goals.
- Issues facilitate **collaboration** by allowing team members to assign tasks, provide feedback, and track progress together.

Labels

- Labels are tags that you can apply to issues and pull requests to categorize them based on various attributes.
- They help you organize your work, filter issues and pull requests, and quickly understand the nature of a particular item.
- Labels provide a way to structure and categorize your project workload.
- You can easily filter issues and pull requests based on specific labels to focus on particular types of work (e.g., all open bugs, all frontend tasks).

Label Categories and Examples

- **Type:**

- `bug`: For reported defects in the application.
- `feature`: For new functionality being proposed or implemented.
- `enhancement`: For improvements to existing features.
- `task`: For general tasks or chores that need to be done.
- `documentation`: For issues related to writing or updating documentation.

- **Priority:**

- `priority:high`: For critical issues that need immediate attention.
- `priority:medium`: For important issues that should be addressed soon.
- `priority:low`: For less critical issues that can be addressed when resources permit.

Label Categories and Examples

- **Status:**

- `status:to do`: For issues that haven't been started yet.
- `status:in progress`: For issues that are currently being worked on.
- `status:needs review`: For pull requests that are ready for code review.
- `status:blocked`: For issues that are currently blocked by another task or issue.
- `status:done`: For issues that have been completed and verified.

Label Categories and Examples

- **Component:**

- `component:user authentication`: For issues related to login, registration, etc.
- `component:task management`: For issues related to creating, editing, and managing tasks.
- `component:project dashboard`: For issues related to the main project overview page.
- `component:reporting`: For issues related to generating reports.

- **Difficulty:**

- `difficulty:easy`: For relatively simple tasks.
- `difficulty:medium`: For tasks requiring a moderate amount of effort.
- `difficulty:hard`: For complex tasks that will take significant time and effort.

Applying Labels

- When an issue or pull request is created, team members can apply relevant labels to it.
For instance:

- A **newly reported bug in the task management module** might get the labels:

- `type:bug`
- `priority:high`
- `component:task management`
- `status:to do`

- A **pull request implementing a new reporting feature** might get the labels:

- `type:feature`
- `component:reporting`
- `status:needs review`

Milestones

- **Milestone** represents a specific goal or timeframe.
- **Issues** associated with that **milestone** are the individual tasks, features, or bug fixes that need to be completed to achieve that goal or within that timeframe.
 - Think of a **milestone as a container or a target**.
 - The **issues within that container are the steps required to reach the target**.

Example Scenario: Planning for "v1.0 Release" Milestone

- Let say we created a milestone named "v1.0 Release" with a due date of April 30, 2025, and the description:
 - "The initial stable release of the project management application with core features.*
 - The core features include:*
 - Login and registration functions*
 - Task management module*
 - Project dashboard"*

Example Scenario: Planning for "v1.0 Release" Milestone

- **Issue #1: Implement user login and registration functionality**

Open

Issue created in 4 hours by BOON GIIN LEE

Title

Users should be able to create new accounts, log in with their credentials, and log out.

Status



Description

Drag your designs here or [click to upload](#).

Milestone

Labels

Add a to do



0 Assignees

Edit

None - assign yourself

Labels

Edit

component:user authentication x

priority:high x

type:feature x

Milestone

Edit

v1.0 Release

Weight

This feature is locked. [Learn more](#)

Due date

Edit

None

Time tracking



Example Scenario: Planning for "v1.0 Release" Milestone

- **Issue #5: Task Management Module:**
 - **Title:** Develop the core task management module.
 - **Description:** Users should be able to create, edit, assign, and track tasks within projects.
 - **Labels:** `type:feature, component:task management, priority:high`
 - **Milestone:** v1.0 Release
 - **Status:** In Progress

Example Scenario: Planning for "v1.0 Release" Milestone

- **Issue #8: Design Project Dashboard:**

- **Title:** Design the main project dashboard layout.
- **Description:** Create wireframes and mockups for the project dashboard, displaying key information and navigation.
- **Labels:** `type:feature, component:project dashboard, priority:medium`
- **Milestone:** v1.0 Release
- **Status:** Open

Tracking of Milestones

Open 1 Closed 0 All 1

v1.0 Release

Mar 20, 2025–Apr 30, 2025

Upcoming BOON GIIN LEE / Team Sheldon

3 Issues · 0 Merge requests

0% complete



after Issue #5 is closed

Open 1 Closed 0 All 1

v1.0 Release

Mar 20, 2025–Apr 30, 2025

Upcoming BOON GIIN LEE / Team Sheldon

3 Issues · 0 Merge requests

33% complete

Benefits of Milestones + Issues

- **Clear Roadmap:** Everyone can see which issues are planned for the `v1.0` release by looking at the milestone.
- **Focused Effort:** The team knows to prioritize issues assigned to the current release milestone.
- **Easy Reporting:** Project managers or stakeholders can quickly check the progress of the `v1.0` release by looking at the milestone's completion status.
- **Release Management:** When it's time to release `v1.0`, you can easily identify all the features and fixes that were included by looking at the closed issues within the "v1.0 Release" milestone.

Summary

- Importance of commit message conventions which various consistencies through **standardized formatting and content**.
- **Issues** are a crucial component of modern Git workflows, providing a **structured platform** for teams to collaboratively manage tasks and problems throughout a project lifecycle.
- **Milestones** are **time-bound goals** that help you track progress towards releases or specific objectives.
- **Labels** are **categorization tags** that provide a flexible way to organize, filter, and understand the nature of issues and pull requests.
- The **association between milestones and issues** is fundamental for effective project management within a Git repository hosting platform.



THANK

YOU